```python
import streamlit as st
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import joblib
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 1: Configure the Streamlit Page (Layout and Title)
st.set_page_config(page_title="Heart Disease KNN demo", layout="wide")
st.title("Heart Disease Prediction - KNN model")
st.success("**🎨 Style Note:** You can customize this application's look and feel! Experiment with different layouts, colors, and Streamlit components to make it your own. Also in the Hackathon!")

# Step 2: Load the persisted models and data
knn_model = joblib.load("knn_model.pkl")
preprocessor = joblib.load("preprocessor.pkl")
dataset = pd.read_csv("heart disease dataset.csv")
results_df = pd.read_csv("heart_disease_actual_vs_predicted_knn.csv")

FEATURES = [
"age","sex","cp","trestbps","chol","fbs","restecg",
"thalach","exang","oldpeak","slope","ca","thal"
]

# Step 3: Organize content into Tabs
tab0, tab1, tab2, tab3 = st.tabs(["Project Workflow", "Real Dataset", "Model Result", "New Prediction"])

# Section: Project Workflow (Methodology)
with tab0:
st.header("Machine Learning Workflow")
st.write("This project follows a systematic approach to predict heart disease using the K-Nearest Neighbors (KNN) algorithm.")
st.subheader("Data Analysis & Preprocessing")
st.markdown("""
- **Data Loading**: Collected heart disease data containing some features.
- **Feature Scaling**: Implemented `StandardScaler` to normalize numerical data, ensuring that features with larger ranges (like cholesterol) don't disproportionately influence the distance-based KNN model.
- **Data Splitting**: Divided the processed data into an 80% training set and a 20% test set for evaluation.
""")
st.subheader("Model Selection & Optimization")
st.markdown("""
- **Algorithm**: Selected K-Nearest Neighbors (KNN) for its effectiveness in classification tasks.
- **Optimal K Search**: Performed a loop iterating through K values from 1 to 21. High-accuracy values were plotted to identify the 'elbow' or peak accuracy point.
- **Best K**: Identified that **K=6** provided the highest accuracy (approx 93.4%) on the test data.
""")
```

```
st.subheader("Evaluation & Persistence")
st.markdown("""
- **Evaluation**: Validated the final model using a Confusion Matrix and Classification Report (Precision, Recall, F1-Score).
- **Persistence**: Saved the trained model (`knn_model.pkl`) and the preprocessor transformer (`preprocessor.pkl`) to bridge the gap between training and this interactive application.
""")

st.subheader("Deployment & Interactive UI")
st.markdown("""
**Setup & Infrastructure:**
1. **Environment Setup**: Ensure `streamlit`, `pandas`, `joblib`, and `scikit-learn` are installed.
2. **Model Persistence**: Load the `.pkl` files created during the training phase.
3. **UI Layout**: Use `st.tabs` and `st.sidebar` to organize the user experience.

**Application Features:**
1. **Display Original Data**: Use `st.dataframe` to share the raw dataset for initial exploration.
2. **Display Evaluation Result**: Visualize the model performance using `st.metric` for accuracy and `st.pyplot` for the confusion matrix.
3. **Prediction Interface**: Build an interactive form using `st.number_input` and `st.selectbox` to allow users to add new patient data and check results.
""")
st.info("💡 **Streamlit Requirement:** This app requires the matching `knn_model.pkl` and `preprocessor.pkl` files in the same directory to function.")

# Section: Real Dataset (Exploration)
with tab1:
st.header("Original Heart Disease Dataset")
st.write(f"Total records: {len(dataset)}")
st.write(f"Total Features: {len(FEATURES)}")
st.dataframe(dataset)

# Section: Model Result (Evaluation)
with tab2:
st.header("Model Performance on Test Set")
accuracy = accuracy_score(results_df["Actual"], results_df["Predicted"])
correct = (results_df["Predicted"] == results_df["Actual"]).sum()
incorrect = len(results_df) - correct

col1, col2, col3, col4 = st.columns(4)
col1.metric("Accuracy: ", f"{accuracy:.1%}")
col2.metric("Total Samples", len(results_df))
col3.metric("Correct Predictions", correct)
col4.metric("Incorrect Predictions", incorrect)

st.subheader("Confusion Matrix & Classification Report")
col_cm, col_cr = st.columns(2)

cm = confusion_matrix(results_df["Actual"], results_df["Predicted"])
report = classification_report(results_df["Actual"], results_df["Predicted"], output_dict=True)
```

```
with col_cm:
st.header("Confusion matrix")
fig, ax = plt.subplots()
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", ax=ax)
ax.set_xlabel("Predicted")
ax.set_ylabel("Actual")
st.pyplot(fig)

with col_cr:
st.header("Classification Report")
report_df = pd.DataFrame(report).transpose().round(2)
st.dataframe(report_df)

# Section: New Prediction (Inference)
with tab3:
st.header("Predict Heart Disease for a New Patient")
st.write("Enter patient data manually or use the example values.")

example_patient = {
"age": 56, "sex": 1, "cp": 0, "trestbps": 132, "chol": 184,
"fbs": 0, "restecg": 0, "thalach": 105, "exang": 1,
"oldpeak": 2.1, "slope": 1, "ca": 1, "thal": 1
}

user_input = {}
for feature, example_val in example_patient.items():
if feature in ["sex","fbs","exang"]:
user_input[feature] = st.selectbox(feature, [0,1], index=example_val)
elif feature == "oldpeak":
user_input[feature] = st.number_input(feature, value=float(example_val), step=0.1)
else:
user_input[feature] = st.number_input(feature, value=int(example_val), step=1)

# Process the Prediction when the user clicks the button
if st.button("Predict"):
# Convert input to DataFrame for transformer
patient_df = pd.DataFrame([user_input])
# Apply the preprocessor (scaling)
patient_scaled = preprocessor.transform(patient_df)
# Get result from KNN model
prediction = knn_model.predict(patient_scaled)[0]
proba = knn_model.predict_proba(patient_scaled)[0][1] if hasattr(knn_model, "predict_proba") else
None

st.subheader("Prediction Result")
if prediction == 1:
st.error("Heart Disease Detected")
else:
st.success("No Heart Disease")
```

```
if proba is not None:
st.info(f"Model confidence (probability of class 1): **{proba:.2f}**")
st.bar_chart({"No Disease": 1-proba, "Heart Disease": proba})

with st.expander("Show entered values"):
st.dataframe(patient_df)
```